Principal Schur specialization and GL2 plethysm in logarithmic space

fengqiaoqu

22 October 2026

Abstract

For an arbitrary partition λ and unary integers $k, r \geq 0$, we show that $[q^r]s_\lambda(1, q, \ldots, q^k)$ is computable in deterministic logarithmic space, with binary output. The same holds for two-row GL_2 plethysm coefficients with an arbitrary outer partition, and for Hermite coefficients. In particular these functions belong to $\mathrm{FL}_{\geq 0} \subseteq \#\mathrm{L}$. The proof combines carry-free integer substitution, the hook-content formula, and the Hesse–Allender–Barrington integer-arithmetic theorem. It also permits binary encoding of k when the partition and r remain unary. This gives affirmative answers to Questions 6.4, 6.22, and 6.29 of the fixed v1 PDF of *Counting in logarithmic space*. The result is an application of classical tools; external review and priority assessment remain outstanding.

1 Statements and conventions

For a partition $\lambda = (\lambda_1 \geq \cdots \geq \lambda_\ell > 0)$ of size $w = |\lambda|$, put

$$S_{\lambda,k}(q) = s_\lambda(1, q, \ldots, q^k) = \sum_{t \geq 0} c_t(\lambda, k) q^t.$$

The empty partition is allowed. Initially, the parts and the nonnegative integers k, r are unary, with fixed separators; L denotes total input length. Invalid encodings have value zero. There is no constant bound on $\ell(\lambda)$. Outputs are binary, without leading zeros except for the output 0.

We use FL for deterministic transducers with read-only input, $O(\log L)$ work space, and write-only output. All transducers constructed below have polynomial time and output length. The notation $\mathrm{FL}_{\geq 0}$ restricts to nonnegative integer-valued functions. The class #L counts accepting paths of polynomial-time nondeterministic logarithmic-space machines.

Theorem 1. *The unary-input function $(\lambda, k, r) \mapsto c_r(\lambda, k)$ belongs to $\mathrm{FL}_{\geq 0} \subseteq \#\mathrm{L}$.*

For $\mu \vdash w$, define plethysm coefficients by $s_\mu[s_{(k)}] = \sum_{\nu \vdash wk} a^\nu_{\mu[(k)]} s_\nu$.

Theorem 2. *For unary input and $0 \leq 2r \leq wk$, the function $(\mu, k, r) \mapsto a^{(wk-r,r)}_{\mu[(k)]}$ belongs to $\mathrm{FL}_{\geq 0} \subseteq \#\mathrm{L}$, without a bound on $\ell(\mu)$. Assign value zero outside this range.*

Theorem 3. *The conclusions of Theorems 1 and 2 remain true if k is binary and the partition and r remain unary. In Theorem 2, the target shape is specified implicitly by (μ, k, r).*

2 Arithmetic and virtual inputs

Our arithmetic input is the theorem of Hesse, Allender, and Barrington [2, Corollary 6.3]: integer division and iterated multiplication are in DLOGTIME-uniform TC^0. In particular, a polynomial number of explicitly represented binary factors of polynomial total length can be multiplied, and two such binary integers can be divided, by deterministic logarithmic-space bit algorithms.

To see the space consequence, evaluate a single output of a uniform constant-depth threshold circuit by recursive recomputation. Each gate level needs a gate index, an input index, and a counter of satisfied inputs; the latter computes a threshold gate. Uniformity generates the wiring and gate types. There are only constantly many levels and each index or counter uses logarithmic space in the arithmetic input length. The circuit has polynomial size and constant depth, so recomputation also takes polynomial time. Repeating for each output bit gives a transducer.

The 2014 corrigendum [3] retracts parts 1 and 3 of Corollary 6.6, which concern bounded arithmetic. It does not retract the integer-arithmetic theorem used here. Neither those retracted statements nor the polynomial-arithmetic corollary is needed.

Lemma 4 (Virtual-input composition)**.** *A constant number of logarithmic-space transducers with polynomially bounded intermediate lengths can be composed in logarithmic space and polynomial time.*

Proof. To simulate $F(G(x))$, store the virtual input-head position j of F. When F reads position j, rerun G using an output-position counter until the requested bit is reached. Polynomially bounded positions need $O(\log L)$ bits. The simulation state and the generating subroutine use additive work space. Their times and the number of recomputations are polynomial. A fixed number of such layers preserves these bounds, without storing an intermediate output in full. □

Lemma 5 (Carry-free substitution)**.** *Suppose* $P(q) = \sum_{t=0}^{d} u_t q^t$ *and* $0 \le u_t < 2^W$*. For* $B = 2^W$*,*

$$u_r = \left\lfloor \frac{P(B)}{2^{Wr}} \right\rfloor \bmod 2^W. \tag{1}$$

If the relevant lengths and positions are polynomially bounded and the bit function of $P(B)$ *is computable in logarithmic space, so is the binary output* u_r*.*

Proof. The sum $\sum_t u_t B^t$ is already a base-B expansion. Thus u_r is the width-W binary block starting at position Wr, numbering the least significant bit by zero. Scan this block to find its highest nonzero bit, then output downwards from that position, or output zero if no such bit exists. Only counters and the bit algorithm are needed. □

3 Principal specialization

Proof of Theorem 1. Repeated scans with counters validate the input and compute w, k, r and $\ell(\lambda)$. If $w = 0$, then $S_{\lambda,k} = 1$. If $\ell(\lambda) > k + 1$, then $S_{\lambda,k} = 0$. If $r > wk$, its coefficient is zero. These boundary cases are immediate. Otherwise $w > 0$ and, with $K = k + 1$, we have $\ell(\lambda) \le K$. Set

$$W = 1 + w\lceil \log_2 K \rceil, \qquad B = 2^W, \qquad b(\lambda) = \sum_i (i-1)\lambda_i. \tag{2}$$

A strict coefficient bound. The tableau expansion of a Schur polynomial has nonnegative coefficients. Its value at $q = 1$ counts semistandard tableaux with K available entries. Counting all assignments to the w boxes gives

$$\sum_t c_t(\lambda, k) = s_\lambda(1^K) \le K^w \le 2^{w\lceil \log_2 K \rceil} < B. \tag{3}$$

This does not assume a length bound. When $K = 1$, it gives $W = 1$ and still holds. Every coefficient therefore satisfies Lemma 5.

Exact integer evaluation. For each box $u = (i, j) \in [\lambda]$, put

$$a_u = K + j - i, \qquad h_u = \lambda_i - j + \lambda'_j - i + 1.$$

The principal-specialization hook-content formula, also used in [1, §6], states

$$S_{\lambda,k}(q) = q^{b(\lambda)} \frac{\prod_u (1 - q^{a_u})}{\prod_u (1 - q^{h_u})}.$$

Since $\ell(\lambda) \le K$, the exponents satisfy

$$1 \le a_u \le K + w, \qquad 1 \le h_u \le w. \tag{4}$$

At $q = B > 1$, cancel the signs of the w numerator and w denominator factors to obtain

$$S_{\lambda,k}(B) = \frac{B^{b(\lambda)} \prod_u (B^{a_u} - 1)}{\prod_u (B^{h_u} - 1)}. \tag{5}$$

The denominator is positive, and the quotient is an exact integer: it is the value of an integer-coefficient polynomial at an integer. Only integer multiplication and division are required.

Polynomial binary length. Unary encoding gives $w, k, r = O(L)$. Also $b(\lambda) \le w^2$ and, using a deliberately coarse bound, $W = O((L + 1)^2)$. The numerator in (5) has bit length at most

$$\begin{aligned} 1 + W\left(b(\lambda) + \sum_u a_u\right) &\le 1 + W\big(w^2 + w(K + w)\big) \\ &= O((L + 1)^4). \end{aligned} \tag{6}$$

The denominator has at most $1 + Ww^2$ bits; the quotient is no longer than the numerator. The integers may be exponentially large in value, but their binary representations have polynomial length.

Generating factors and extracting a block. A stored box coordinate (i, j) and a scan determine λ_i. Another scan counts the parts at least j to obtain λ'_j. All exponents, positions, and quantities in (2) and (4) are polynomially bounded and computable with a constant number of logarithmic-size counters.

Each factor $B^e - 1 = 2^{We} - 1$ is a string of We ones. The factor $B^{b(\lambda)}$ is a one followed by $Wb(\lambda)$ zeros. Consequently, the factor lists in (5), including padding and separators for a fixed arithmetic encoding, are polynomial-length virtual inputs with logarithmic-space bit functions. Apply iterated multiplication separately to the two lists and integer division to the products. By Lemma 4, the bit function of $S_{\lambda,k}(B)$ is in logarithmic space. There are constantly many composition layers: the number of boxes is the number of factors in one iterated-multiplication instance, not the number of layers. Finally (3) and Lemma 5 compute c_r in FL.

A counting machine. For the #L inclusion, use $0 \le c_r < 2^W$. Guess exactly W bits, most significant first, specifying $y \in \{0, \dots, 2^W - 1\}$. Maintain a position counter and the three-way comparison state of the prefixes of y and c_r. At each position, deterministically recompute the corresponding bit of c_r, padded with zeros. Accept exactly when $y < c_r$. There is one path per length-W binary string and hence exactly c_r accepting paths. The machine has logarithmic work space and polynomial time on every path. Each guessed bit is consumed once; past nondeterministic choices are not guessed again to reconstruct a witness. Boundary cases use the same construction or machines with zero or one accepting path. □

4 Plethysm and Hermite coefficients

Proof of Theorem 2. Evaluating the plethysm alphabet gives

$$s_\mu[s_{(k)}](1, q) = s_\mu(1, q, \dots, q^k),$$

because $s_{(k)}(1, q) = 1 + q + \cdots + q^k$. Schur polynomials with more than two rows vanish in these two variables, while

$$s_{(wk-t,t)}(1, q) = q^t + q^{t+1} + \cdots + q^{wk-t} \quad (0 \le 2t \le wk).$$

For $0 \le 2r \le wk$, comparison of adjacent coefficients gives

$$a^{(wk-r,r)}_{\mu[(k)]} = c_r(\mu, k) - c_{r-1}(\mu, k), \qquad c_{-1} = 0. \tag{7}$$

Both terms are in FL. Subtraction is in logarithmic space: to output a bit, rescan less significant positions to recompute its borrow, keeping a single borrow bit. Compose this with the coefficient bit algorithms using Lemma 4. The difference is nonnegative because it is a representation multiplicity, and it is at most $c_r < 2^W$. The fixed-length comparison machine in the preceding proof puts it in #L. No closure of #L under subtraction is used, and no bound on $\ell(\mu)$ was imposed. □

For $\mu = (n)$, let $p_r(n \times k)$ count partitions of r contained in an n-by-k rectangle. Then

$$s_{(n)}(1, q, \dots, q^k) = \begin{bmatrix} n+k \\ k \end{bmatrix}_q = \sum_r p_r(n \times k) q^r.$$

The Hermite identity [1, (6.1)–(6.2)] yields

$$a^{(nk-r,r)}_{(n)[(k)]} = g((nk - r, r), (n^k), (n^k)) = p_r(n \times k) - p_{r-1}(n \times k) \tag{8}$$

for $0 \le 2r \le nk$, where (n^k) has k parts equal to n. Empty cases are interpreted by the constant Schur polynomial. These coefficients are therefore in $\mathrm{FL}_{\ge 0} \subseteq$ #L.

There is also a shorter special-case size bound. Set $W = n+k+1$, as $\sum_r p_r(n \times k) = \binom{n+k}{k} < 2^W$. At $q = 2^W$,

$$\begin{bmatrix} n+k \\ k \end{bmatrix}_{2^W} = \frac{\prod_{i=1}^{k} (2^{W(n+i)} - 1)}{\prod_{i=1}^{k} (2^{Wi} - 1)}.$$

For $k > 0$, the numerator and denominator have at most $Wk(2n + k + 1)/2$ and $Wk(k + 1)/2$ bits, respectively. If $k = 0$, both products are empty and the quotient is one.

5 A mixed-encoding extension

Proof of Theorem 3. For every $0 \le t \le r$, the tableau expansion shows

$$[q^t]s_\lambda(1, q, \dots, q^k) = [q^t]s_\lambda(1, q, \dots, q^{\min(k,r)}). \tag{9}$$

A tableau using a letter of weight q^j for $j > r$ has total degree greater than r, since all other weights have nonnegative degree. Tableaux using only retained letters satisfy exactly the same row and column conditions. This proves the claimed bijective restriction, including cases where the truncated alphabet is too short for the shape.

While reading binary k, maintain a saturated counter updated by $z \leftarrow \min(r, 2z + b)$ for the next bit b. This computes $\min(k, r)$ in logarithmic space. Its virtual unary representation has polynomial length because r is unary. Theorem 1 and Lemma 4 apply, proving the coefficient claim. The same truncation preserves both coefficients in (7).

For the validity condition of Theorem 2, if $w > 0$ compute $\widehat{k} = \min(k, 2r + 1)$ by saturation. Then $2r \le wk$ if and only if $2r \le w\widehat{k}$, and the latter comparison uses polynomially bounded integers. If $w = 0$, validity requires $r = 0$. Thus the implicit target shape can be checked without storing a large binary product. This extension does not encode all parameters in binary. □

6 Relation to the stated questions and limitations

The reference for numbering is the fixed arXiv v1 PDF of Gutiérrez, Ikenmeyer, and Panova [1], dated 17 July 2026. Its experimental HTML rendering has different labels:

v1 PDF question	Page	HTML label	Consequence here
6.4	28	6.4	Hermite coefficients are in $\mathrm{FL}_{\ge 0} \subseteq \#\mathrm{L}$.
6.22	36	6.21	Principal specialization, with arbitrary partition length.
6.29	38	6.27	Two-row GL_2 plethysm, with arbitrary outer length.

For Question 6.29, $\#\mathrm{L} \subseteq \#\mathrm{TISP}(\mathrm{poly}(L), \log^2 L)$ gives the requested inclusion. These are three consequences of one main coefficient theorem. The mixed-encoding extension is not counted as a separately sourced open question.

The argument uses a single-variable polynomial whose exact integer evaluation has polynomial binary length. It does not give an algorithm of this kind for general Littlewood–Richardson or Kronecker coefficients, and does not resolve whether general Kronecker coefficients belong to #P or separate them from that class.

The accompanying Python checks compare exact integer substitution with independent tableau and rectangle-partition computations. They check finite values and identities; they do not implement the logarithmic-space multiplication and division transducers. The space theorem follows from the proof and [2], not from the memory use of those programs.

Research-status notice. This draft received internal AI-assisted mathematical review and finite exact checks. It has not received external peer review or author confirmation, and is not a Lean formalization. No global priority claim is made. The central observation applies classical formulas and integer-arithmetic results to the stated coefficient functions.

References

[1] Álvaro Gutiérrez, Christian Ikenmeyer, and Greta Panova. *Counting in logarithmic space*. arXiv:2607.15881v1, 17 July 2026. Fixed v1 PDF; v1 HTML.

[2] William Hesse, Eric Allender, and David A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *Journal of Computer and System Sciences* **65**(4), 695–716, 2002. Author PDF.

[3] William Hesse, Eric Allender, and David A. Mix Barrington. Corrigendum for Uniform Constant-Depth Threshold Circuits for Division and Iterated Multiplication. *Journal of Computer and System Sciences* **80**, 496–497, 2014. Author PDF.